BUILD LOG

Building VC Whisper Prepare as a Claude Code Skill

March 26, 2026

Conner Ruhl directing Claude Code via voice input (SuperWhisper)

Single evening session, start to finish

github.com/cruhl/vc-prepare

This is the raw build log of a Claude Code session. The entire thing was directed by voice (SuperWhisper speech-to-text), so what follows is cleaned-up dictation, not polished copywriting. The decisions are real, the quotes are real, the frustrations are real. What follows is the chronological record of a single Claude Code session.

1

The Spark 6:15 PM

> *"I would, that's how I do literally everything nowadays. Custom CLI or skill. Just ask Claude to make you the skill."*

Parth Chopra had just launched **VC Whisper Prepare** (vcwhisper.com/prepare) and was interested in making it a Claude Code skill. Rather than explain how skills work, Conner decided to just build it and hand over the repo.

2

Research Phase 6:17 PM

Three parallel research agents launched simultaneously, each attacking a different axis of the problem:

PARALLEL RESEARCH AGENTS

- Scrape vcwhisper.com/prepare for the full site analysis
- Research Claude Code skills documentation from Anthropic
- Examine existing skills in `~/.claude/skills/` for patterns

The scraper found the complete API structure: `/api/prepare` for the main analysis endpoint, `/api/prepare/search-investors` for autocomplete, `/api/prepare/parse-deck` for PDF parsing, and `/api/prepare/veck-lookup` for the investor database. Under the hood, the tech stack was Exa, Seedlist, xAI, Browserbase, and (in Parth's words) "a lot of prompt engineering."

3

Architecture Decisions 6:25 PM

Three approaches on the table:

APPROACH A

Pure Skill with WebSearch Only

Zero configuration. Relies entirely on Claude's built-in web search. Roughly 75% quality compared to the full product.

APPROACH B

Wrap Parth's API

Identical quality to the web app. Hard dependency on Parth's service staying up and maintaining the same endpoints.

APPROACH C CHOSEN

Hybrid: WebSearch Default, API Keys Optional

Zero friction install that works immediately. Premium tier unlocked when users add their own API keys for Exa, xAI, or Browserbase enrichment.

The decision was straightforward. Zero friction install by default, but if you have API keys, you get the premium tier.

4

The Build 6:35 PM

A new git repo went up at `/Users/cruhl/GitHub/vc-prepare`. The initial build produced four core files:

- **SKILL.md** with frontmatter, research instructions, and the output template
- **README.md** with install instructions, usage examples, and API key docs
- **LICENSE** (MIT, under Parth's name)
- An example output for **Paul Graham**

Pushed to GitHub as `cruhl/vc-prepare`. A functional skill, live, in one pass. But nowhere near done.

5

Feature Parity Audit 6:42 PM

> *"I want another agent to strip-mine the website for a gigantic list of every functional requirement and feature, how data is outputted, the formatting, etc. So that we can adversarially prompt to make sure that we have complete feature parity."*

The skeleton was up, but there was no confidence it actually matched what Parth's web app does. A dedicated agent launched to adversarially compare the skill's output against every feature the web product offered.

AUDIT RESULTS

- 9 input fields, 4 top-level output sections, 9 subsections
- Complete data schema with all fields documented
- Loading states, error handling, example data flow
- Color palette, typography, and layout patterns

This became the specification for the next round of work. If the web product had it, the skill needed to support it.

6

Architecture Overhaul 6:50 PM

> *"Basically, if Claude writes the JSON, we should be able to have a perfect report every time, and that's the sort of main final output. We should also have a beautiful CLI output version of the same document in case they don't give a shit about the PDF."*

The initial build was functional but structurally naive. The pipeline needed to be rethought from scratch. Conner's key insight: separate data from presentation entirely.

- **JSON-first pipeline:** Research produces structured JSON first, then the CLI and PDF renderer consume it independently
- **All fields optional:** Graceful degradation when data is unavailable
- **Parallel subagent research:** Four agents (career, portfolio, tweets, industry context) running simultaneously
- `schema.json` **as the single source of truth:** One schema governs the data contract across every layer

The first pass was a prototype. This was the product.

7

The PDF Renderer

7:00 PM

> *"Think $2,000 minimum paid report by a consultant who specializes in this activity."*

`render.py` came in at 1,200 lines of Python with a fully embedded HTML/CSS template. The bar was set from the first sentence. Design choices included Cormorant Garamond for headings, Source Serif for body text, Inter for labels, and a warm earth-tone palette matching VC Whisper's existing aesthetic. Cover page with investor name, fit score badge, and metadata.

> *"We need no headers or footers when we actually output the PDF."*

> *"The page breaks are not perfect. They need to be perfect. We cannot split stuff naively."*

The first version did not meet the standard:

> *"It's just kind of an embarrassing level of Claude Code smelling mess at the moment. Think about what we can improve and the best fit for print media."*

The conversion pipeline was built with a three-step cascade: try WeasyPrint first, fall back to wkhtmltopdf, then Chrome headless as the last resort. Redundancy over fragility.

8

API Verification

7:15 PM

> *"Make sure everything works with actual API keys and Twitter results, etc. And think about if you are Parth and you're just trying this out for the first time, exactly what you would need to do."*

Every API integration was verified against current documentation, with the mindset of someone trying this for the first time. The results were mixed:

VERIFICATION RESULTS

- **Exa:** Correct as written
- **xAI:** Three errors fixed. Grok-3 was deprecated; switched to the Responses API with the `x_search` tool
- **Seedlist:** Completely wrong. No API key needed; it's a free static JSON endpoint
- **Browserbase:** Auth header documented correctly (`X-BB-API-Key`)

Two out of four integrations had errors that would have caused silent failures in production.

9

The Meta Example 7:25 PM

> *"The default example pitch and company and idea should be kind of tongue-in-cheek but keep it realistic. VC Prepare itself as an idea, as if you were pitching VC Prepare using VC Prepare. I think that would be pretty funny."*

The default example became VC Prepare pitching itself to Paul Graham. Self-referential, but also a genuine end-to-end test of every pipeline stage. If the tool can analyze its own product for its own target investor, the pipeline works.

10

Founder Interview Stage 7:30 PM

A structured interview was added before any research begins. The skill now asks three foundational questions:

1. What does your company do?
2. What is the company website?
3. What stage are you at, and how much are you raising?

Then one to two targeted follow-ups based on the answers: traction metrics, defensible moat, biggest anticipated objection, or any prior contact with the target investor. Context before research. The skill asks before it acts.

> *"I do not want the examples to be handwritten. They need to be generated as if the skill was used itself."*

No faking it. The examples are real output from the skill running against real data.

11

Polish 7:35 PM

> *"Make sure to say this was made for Parth. Like, it just seems too much like my idea at the moment."*

> *"Write the README as if you were that person trying it for the first time, but don't say that directly."*

The cleanup covered every surface:

- Tweet screenshots embedded in the README for social proof
- PDF preview image generated from the Paul Graham example
- README compressed from roughly 130 lines down to 75
- Renamed to "VC Whisper Prepare" to match Parth's branding exactly
- `generate-examples.sh` rewritten with detailed pipeline instructions

No features were added. Everything was tightened, trimmed, and aligned. The last 10% of polish is what separates a prototype from something you can hand to someone and feel good about.

12

The Build Log Itself 7:45 PM

> *"I want an extremely well-formatted, beautiful HTML document that I can print that basically is as close as possible to the words I'm saying, but accounts for the fact it's text-to-speech, so it's choppy. Maybe cleaned up a little bit, maybe summarized a little bit."*

> *"Don't be afraid to include my cussing. Just don't make me sound like an idiot. Make sure my words are well formatted. Make sure the general essence of what I'm saying is very well communicated."*

Toward the end of the session, Conner turned the build process on itself. The document you are reading was produced as part of the same Claude Code session it describes. Voice dictation captured throughout the evening was cleaned, structured, and rendered into this timeline. The constraint was fidelity: preserve the actual words, account for the choppiness of speech-to-text, but do not sanitize or editorialize.